

Medium Day is Sept 18. Sign up for free and discover something new! [Register now.](#)



Robotics



Inverse Kinematics



Simulation



Control System



Python



When More Damping Makes Your IK Solver Worse

A non-monotonic result in differential inverse kinematics, and three wrong explanations before the right one.



Luminolous

[Follow](#)

10 min read · 3 days ago



47



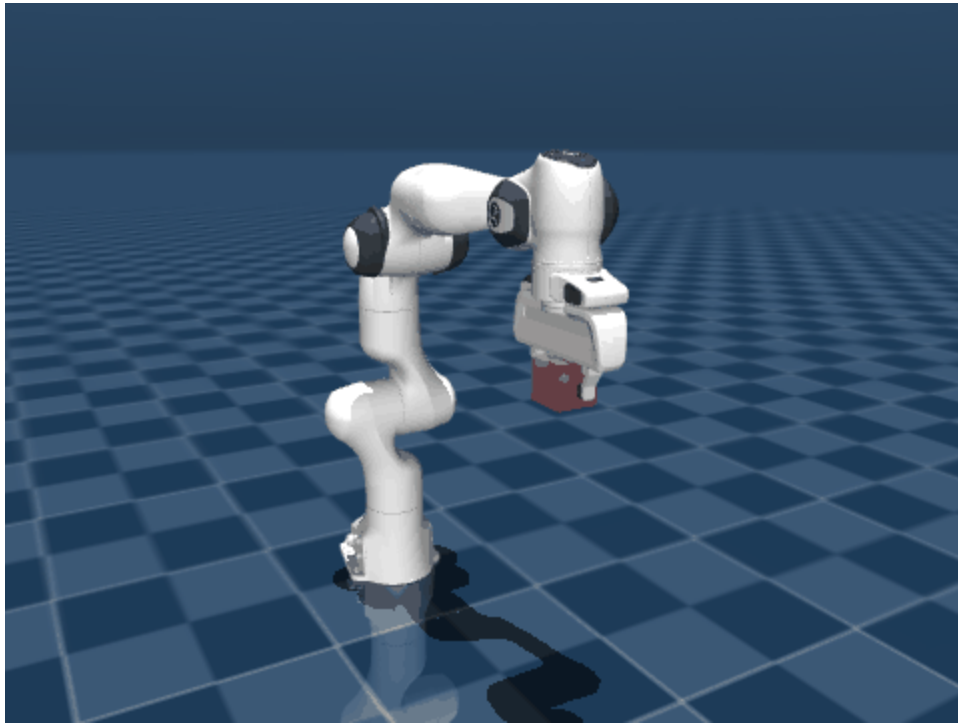
I swept the damping parameter of an inverse kinematics solver across five orders of magnitude, expecting a smooth trade-off between tracking accuracy and stability. I got a curve with a hole punched in the middle of it.

Damping λ	RMS position error [m]	Final position error [m]	Peak cond(J)	Clipped steps
10^{-6}	0.0145	0.0066	40 569	12
10^{-5}	0.0145	0.0066	40 472	12
10^{-4}	0.0145	0.0066	32 681	12
3×10^{-4}	0.0136	0.0066	12 979	10
5×10^{-4}	0.1758	0.4302	128 652	1145
10^{-3}	0.1757	0.4302	693 352	1140
3×10^{-3}	0.1754	0.4301	1 572 330	1140
5×10^{-3}	0.1748	0.4300	1 572 210	1140
7×10^{-3}	0.0089	0.0066	2280	0
10^{-2}	0.0089	0.0066	580	0
10^{-1}	0.0120	0.0066	57	0

Damping sweep at a reach of 0.70 m. Four values in the middle fail by a factor of twenty, and the failure disappears again as the damping keeps rising.

Small damping works. Large damping works. Four values in between fail by a factor of twenty, then the failure vanishes as you keep increasing the same parameter. No textbook description of damped least squares predicts this shape, and my first three explanations for it turned out to be wrong.

The reason to read on, if you work on anything with more joints than task dimensions: the mechanism behind this has little to do with damping, and it applies to any redundant manipulator. If you run whole-body control on a humanoid, you have far more null directions than the one I am about to describe.



The solver tracking a target the user drags through the workspace. Every frame solves one damped least squares problem and hands the result to the position actuators.

What the solver does

The Jacobian relates joint velocity to end-effector velocity:

$$\dot{x} = J(q) \dot{q}$$

For the 7-joint Franka Panda and a 6-dimensional pose task, J is 6×7 . You want the inverse relationship: given a pose error e , which joint velocity shrinks it? With more unknowns than equations the standard answer takes the minimum-norm solution through the pseudo-inverse.

$$\dot{q} = J^+ e, \quad J^+ = J^T (J J^T)^{-1}$$

Straighten the arm out and λ loses rank. The pseudo-inverse then asks for enormous joint velocities to produce a small end-effector motion, and your robot lunges. Damped least squares heads that off by adding a term to the matrix you invert.

$$\dot{q} = J^T (JJ^T + \lambda^2 I)^{-1} e$$

The $\lambda^2 I$ keeps the 6×6 matrix invertible whatever λ does. Far from a singularity, λ^2 contributes almost nothing and you recover the pseudo-inverse. Close to one, it bounds the joint velocity and you pay for that bound in accuracy.

Textbooks describe one knob and one gradient: small λ tracks tightly and misbehaves near singularities, large λ stays smooth and tracks loosely. Table 1 is a different shape.

My test drives a target straight out from the home pose to 0.70 m from the shoulder and back, over 6 seconds at a 2 ms timestep, holding orientation fixed. That radius does a lot of work. The Panda reaches roughly 0.85 m if you only constrain position, but pinning the orientation costs workspace, and past about 0.70 m the requested pose leaves the reachable set. The run sits on that edge by design, because that is where damping has anything to do.

Wrong explanation one: too much damping

The obvious reading says heavy damping sacrifices accuracy. It dies on contact with the table. $\lambda = 10^{-2}$ carries more damping than any failing value and gives one of the best runs in the sweep.

Wrong explanation two: the arm is stuck in a singularity

The failing runs log condition numbers up to 1.57 million, which makes this a reasonable suspicion. Reading the time series kills it. At the moment the run reaches its worst error, $\text{cond}(\mathbf{J})$ has come back down to around 32. The arm sits in a well-conditioned configuration while it fails. Whatever broke happened earlier, and the huge condition number belongs to the aftermath.

Wrong explanation three: joint-limit clipping

I held this one for a while, and watching it collapse taught me more than the answer did.

After solving for $\mathbf{q}$ you integrate it into a position setpoint, then clip that setpoint to the joint limits before commanding it. Every failing run clips for more than 1,100 of its 3,000 steps. Every healthy run clips a dozen times or never. The correlation across the sweep runs above 0.95.

The mechanism sounded right too. Clipping shortens the commanded motion and points it somewhere else, so the arm jams against a limit and cannot work its way free.

Then I ran the ablation, holding $\lambda = 10^{-3}$ and changing one thing at a time.

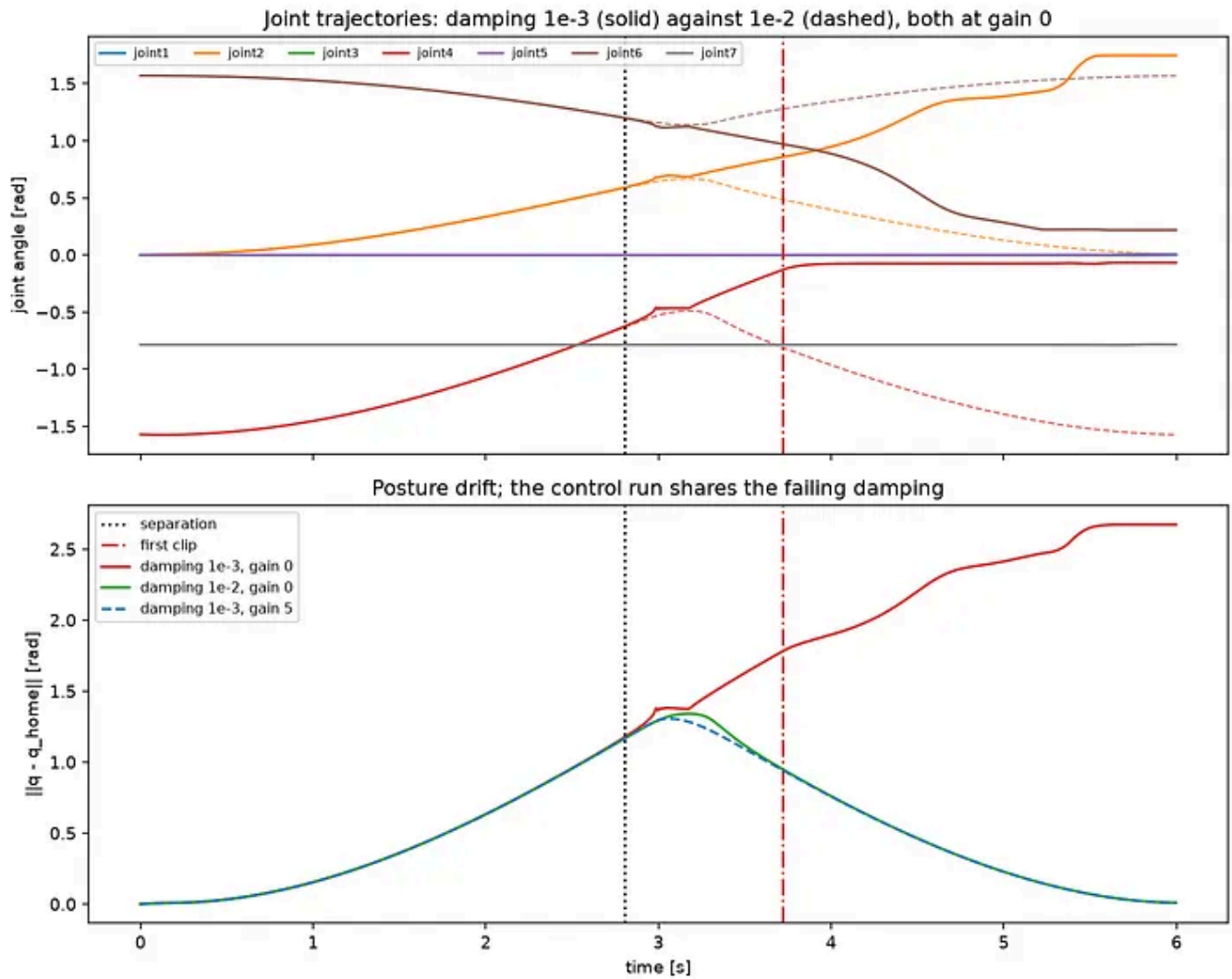
Condition at $\lambda = 10^{-3}$	Final position error [m]	Clipped steps	Drift from home [rad]
Baseline	0.4302	1140	2.673
Velocity clamp, $\ \dot{q}\ \leq 1$	0.4301	1140	2.673
Joint-limit clipping disabled	0.4299	0	2.699
Nullspace, $K_n = 5$	0.0066	0	1.306
Nullspace, clipping disabled	0.0066	0	1.306

Ablation at the damping that fails. Removing the clipping changes the outcome by 0.0003 m. The nullspace term changes it by a factor of sixty-five.

Turning the clipping off repairs nothing. The run lands at 0.4299 m instead of 0.4302 m. Bounding the joint velocity leaves the outcome untouched as well. The only change that rescues the run is a secondary objective projected into the nullspace, and it works with clipping on or off.

If you try to reproduce this, one detail will bite you. MuJoCo clamps `data.ctrl` against each actuator's `ctrlrange` by default, and the Panda's `ctrlrange` matches its joint range. Disable your own clipping without also disabling `mjDSBL_CLAMPCTRL` and you have moved the clipping inside the physics engine rather than removing it, which hands you a null result for the wrong reason.

The timing closes the case. Logging a failing run and a healthy one step by step, their joint trajectories separate at step 1401. The first clip in the failing run happens at step 1860. The two runs had already parted company for 459 steps, almost a full second, before anything got clipped.



Joint trajectories for the failing and healthy runs. The vertical markers show where the paths separate and where the first clip occurs, 459 steps apart.

Where the arm actually ends up

Compare the final configurations joint by joint.

Run	joint2 [rad]	joint4 [rad]	joint6 [rad]	Final position error [m]
$\lambda = 3 \times 10^{-4}$	+0.009	-1.574	+1.569	0.0066
$\lambda = 10^{-2}$	+0.009	-1.574	+1.569	0.0066
$\lambda = 10^{-3}, K_n = 5$	+0.009	-1.574	+1.569	0.0066
$\lambda = 10^{-3}$	+1.747	-0.068	+0.219	0.4302
Joint limit	± 1.763	$[-3.072, -0.070]$	$[-0.018, 3.753]$	

Final joint angles. Three healthy runs agree to three decimals across a 30x range of damping. The failing run rests against two joint limits at once.

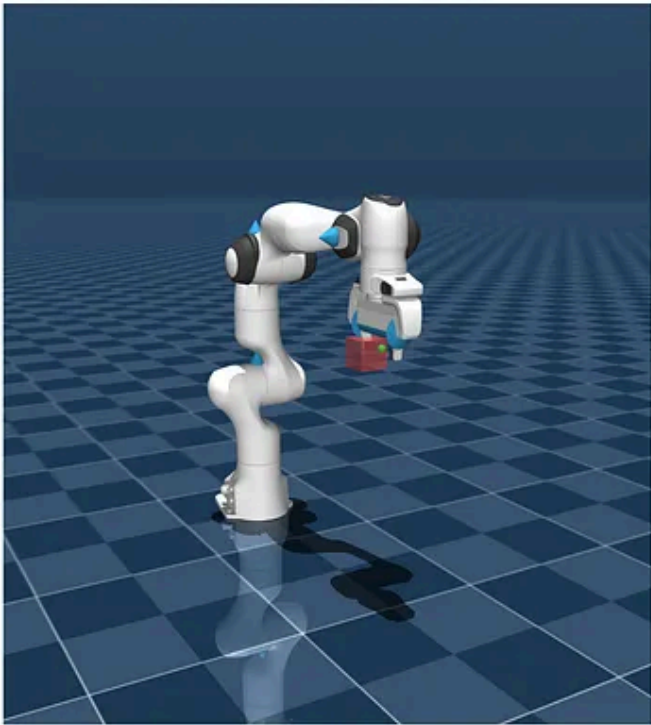
The three healthy runs return to the same posture, to three decimals, across a 30x range of damping. The failing run finishes somewhere else entirely, with joint4 resting at -0.068 rad against an upper limit of -0.0698, and joint2 at 1.747 against a limit of 1.7628. The elbow has straightened and locked, and the shoulder has swung up to compensate. The arm ends folded onto two of its limits at once.

The solver resolved the redundancy into a configuration it could not leave.

Seven joints serving a six-dimensional task leave one direction of joint motion that moves nothing at the gripper. The elbow swings while the hand stays put. With no secondary objective, nothing at all decides where that free coordinate goes: the minimum-norm solution picks whatever is locally cheapest, step by step, and it has no memory. Passing near the singularity, the free coordinate can settle on either side of a fold, and λ decides which side.

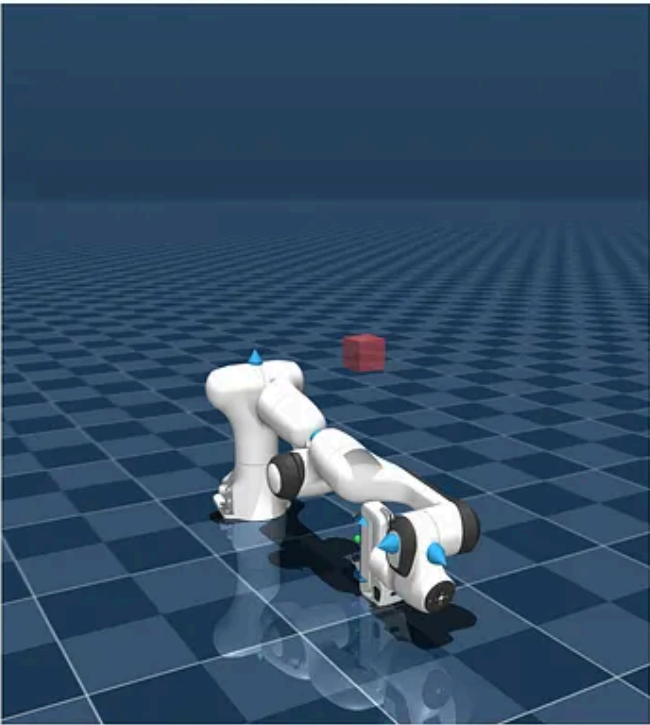
Healthy run

lambda = 1e-3 with nullspace, final error 0.0066 m



Collapsed run

lambda = 1e-3, final error 0.4302 m



Both runs use the same damping and the same target. The one on the right resolved the redundancy into a fold it could not undo, and finished 0.43 m from a target it had already reached once.

Two outcomes with nothing in between

Once I knew what to measure, the structure became easy to see. I ran a grid of 126 runs across six reach radii, seven damping values and three nullspace gains, recording each run’s final distance from the home posture.

End state	Runs	Drift from home [rad]	Spread
Healthy	93	0.000	within 0.001 18 rad of home
Lagging	13	≤ 1.599	target unreachable at peak
Collapsed	20	2.671 to 2.673	within 0.0079 rad of each other

End states across 126 runs. The collapsed group is tighter than the measurement noise on any single joint.

Twenty runs land between 2.671 and 2.673 rad from home. The other 106 stay at or below 1.599. The interval between those clusters holds nothing.

Matching all seven joint angles rather than the norm makes the point sharper. The twenty collapsed runs finish within 0.0079 rad of each other.

Open in app ↗



Medium



Search



Write



healthy runs are just as consistent in the other direction, finishing within 0.00118 rad of home.

A metric that lied to me

That grid also exposed a flaw in my own headline number.

RMS position error mixes two failures that have nothing to do with each other. A run scores badly if it collapses, and it also scores badly if the target was unreachable at the peak, in which case the arm lags through the middle of the trajectory and then comes home fine. Thirteen runs in the grid are the second kind, and at 0.70 m the two coincide closely enough that I never noticed.

Final position error separates them cleanly. All 106 non-collapsed runs finish within 0.0066 m of the target whatever their RMS says. Posture drift separates them too. If you sweep parameters and choose your outcome metric without care, you can spend a week explaining a single phenomenon that is really two.

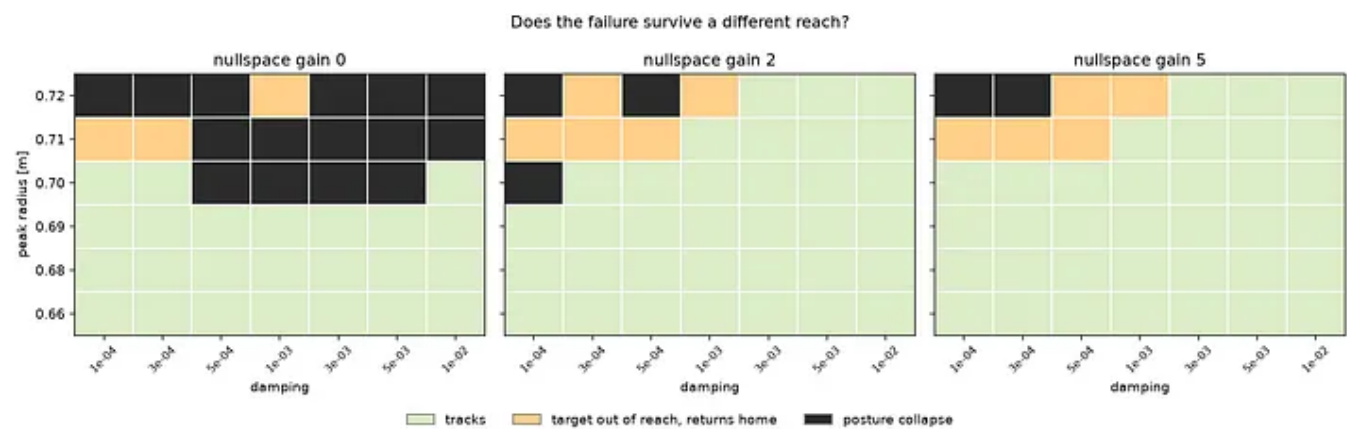
Does any of this survive a different reach?

Here the tidy version of the story dies, and I think the untidy version is worth more.

Peak reach radius [m]	Collapsed runs out of 7 damping values		
	$K_n = 0$	$K_n = 2$	$K_n = 5$
0.66	0	0	0
0.68	0	0	0
0.69	0	0	0
0.70	4	1	0
0.71	5	0	0
0.72	6	2	2

Collapsed runs by reach radius and nullspace gain. Nothing collapses below 0.70 m, and raising the gain shifts the vulnerable cells rather than clearing them.

The collapse does not occur below 0.70 m. It appears at 0.70 and widens as the target reaches further. So the interval $\lambda \in [5 \times 10^{-4}, 5 \times 10^{-3}]$ describes one trajectory, not damped least squares. What holds across radii is the mechanism and the posture, while the numbers move.



The same grid as a map. The collapse region grows with reach instead of sitting at a fixed interval of damping.

The nullspace term offers no general cure either. It clears every collapse at 0.70 m by gain 5, yet at 0.72 m a gain of 5 still collapses at two damping values, and at 0.70 m a gain of 2 introduces a collapse that gain 0 avoids. Raising the secondary gain relocates the vulnerable region.

For reference, I also ran the same trajectory through a QP-based solver, which handles joint and velocity limits as hard constraints. It never collapses. Before crediting the QP formulation for that, note that it also carries a posture task by default, which is the same kind of secondary objective that rescues the DLS side. Its immunity here does not come from the QP alone.

What to take from this

Damping controls conditioning. It has nothing to say about redundancy. These two problems get conflated because both surface near singularities. λ keeps $\mathbb{J}\mathbb{J}^\top$ invertible, and that is the whole of its job. Where the null direction drifts is a separate question, and on a redundant arm that drift decides whether you can come back.

A secondary objective earns its place. I implemented the nullspace projector because it demos well, letting you swing the elbow while the gripper holds still.

$$\dot{q} = J^+ e + (I - J^+ J) \dot{q}_0, \quad \dot{q}_0 = K_n (q_{\text{home}} - q)$$

Setting $\dot{q}_0 = K_n (q_{\text{home}} - q)$ costs almost nothing when the arm starts at home, since it pulls toward the posture the arm already holds. Its real job is keeping the solver inside one basin over a long run.

Correlation near a singularity is worth very little. Clipping tracked failure at better than 0.95 across the sweep and had no part in causing it. The ablation and the step-level timing settled the question. When a parameter sweep

hands you a suspicious pattern, switch the suspected cause off before you write it up.

If you want to check your own solver for this, three things are cheap to log and tell you most of what you need: the distance from a reference posture at every step, the step index of the first joint-limit clip, and the final error separate from the RMS. Run the same target trajectory at two damping values and diff the joint traces. Divergence that shows up before any limit is touched points at redundancy rather than at your clipping code.

Scope

One trajectory shape, radial with orientation held constant. One integration gain, one robot. The 0.70 m threshold belongs to this path rather than to the Panda. I have not tested whether the same fold is reachable at shorter reaches under a rotating target or a curved path, and I would expect that it is.

References

[1] D. E. Whitney, “Resolved motion rate control of manipulators and human prostheses,” *IEEE Transactions on Man-Machine Systems*, vol. 10, no. 2, pp. 47–53, 1969.

[2] Y. Nakamura and H. Hanafusa, “Inverse kinematic solutions with singularity robustness for robot manipulator control,” *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.

[3] C. W. Wampler, “Manipulator inverse kinematic solutions based on vector formulations and damped least-squares methods,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, 1986.

[4] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.

[5] S. Chiaverini, “Singularity-robust task-priority redundancy resolution for real-time kinematic control of robot manipulators,” *IEEE Transactions on Robotics and Automation*, vol. 13, no. 3, pp. 398–410, 1997.

[6] A. Liégeois, “Automatic supervisory control of the configuration and behavior of multibody mechanisms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 7, no. 12, pp. 868–871, 1977.

[7] S. R. Buss, “Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods,” unpublished technical note, University of California San Diego, 2004.

[8] Google DeepMind, “MuJoCo Menagerie: A collection of high-quality models for the MuJoCo physics engine,” 2022. [Online]. Available: https://github.com/google-deepmind/mujoco_menagerie

[9] K. Zakka, “mink: Python inverse kinematics based on MuJoCo,” 2024. [Online]. Available: <https://github.com/kevinzakka/mink>

Code, data and every number quoted here: github.com/luminolous/diffik-panda

Robotics

+

Inverse Kinematics

+

Simulation

+

Control System

+

Python

+



Written by Luminolous

7 followers · 9 following

Deep Learning Researcher

Follow

No responses yet



Alex Mylnikov

What are your thoughts?

